EXPERIMENT 6

Implement Dijkstra's Algorithm for the Single-Source Shortest Path Problem in Directed and Undirected Graphs

Name	HIMANSHU RAJ
Batch	B1 CCVT
SAP ID	590026147
Submitted To	Arjun Abhishek

B.Tech CSE – Cloud Computing & Virtualization (CCVT)

1. Aim

To implement Dijkstra’s algorithm for the single-source shortest path problem in both directed and undirected weighted graphs, and display the shortest distance and path from the specified source vertex.

2. Objectives

• Understand Dijkstra’s greedy shortest-path algorithm.

• Calculate shortest distances from one source to every reachable vertex.

• Implement the algorithm for an undirected weighted graph with source vertex 0.

• Implement the algorithm for a directed weighted graph with source vertex A.

• Display the shortest distance and corresponding path for each vertex.

3. Theory

Dijkstra’s algorithm is a greedy algorithm used to find shortest paths from a single source vertex to all other vertices in a weighted graph with non-negative edge weights. It repeatedly chooses the unvisited vertex with the smallest tentative distance and relaxes its outgoing edges.

In an undirected graph, every edge can be traversed in both directions. In a directed graph, each edge can only be traversed in the direction shown by its arrow. The implementation uses an adjacency matrix and a linear scan to find the next minimum-distance vertex.

4. Algorithm

1. Initialize the distance of every vertex to infinity, mark all vertices unvisited, and set every parent to undefined.

2. Set the source vertex distance to zero.

3. Choose the unvisited vertex with the smallest tentative distance.

4. If no reachable unvisited vertex remains, stop; otherwise mark the selected vertex visited.

5. For each outgoing edge, update a neighbor’s distance and parent if a shorter path is found.

6. Repeat until all reachable vertices have been processed.

7. Print the shortest distance and reconstruct each path using the parent array.

5. Sample Input and Output

A. Undirected graph

Source vertex: 0

Vertex	Shortest Distance	Shortest Path
0	0	0
1	4	0 → 1
2	12	0 → 1 → 2
3	19	0 → 1 → 2 → 3
4	8	0 → 4
5	9	0 → 4 → 5
6	11	0 → 4 → 5 → 6
7	21	0 → 4 → 5 → 6 → 7
8	14	0 → 1 → 2 → 8

B. Directed graph

Source vertex: A

Vertex	Shortest Distance	Shortest Path
A	0	A
B	8	A → C → B
C	5	A → C
D	9	A → C → B → D
E	7	A → C → E

6. Complexity Analysis

Operation	Complexity
Selecting minimum-distance vertex	O(V) per iteration
Relaxing edges with adjacency matrix	O(V) per iteration
Overall time complexity	$O(V^2)$
Auxiliary space	O(V), excluding graph storage

7. Performance Discussion

The implementation uses an adjacency matrix and scans all vertices to find the next minimum tentative distance. This results in $O(V^2)$ time complexity. Dijkstra's algorithm requires non-negative edge weights; it does not correctly handle graphs containing negative-weight edges.

8. Result

Dijkstra's algorithm was implemented successfully for both the undirected graph with source vertex 0 and the directed graph with source vertex A. The program displayed the shortest distances and paths from each source, matching the results obtained in the terminal.

9. Screenshots / Evidence

The following screenshots document the implementation and successful execution in Visual Studio Code. Source code is omitted from this report, consistent with the previous experiment reports.

Screenshot 1

exp5.c > ...

```c

#include <stdio.h>

#define MAX 9
#define INF 999999

// Print the shortest path recursively
Windsurf: Refactor | Explain | ×
void printPath(int parent[], int v, char labels[]) {
    if (parent[v] == -1) {
        printf("%c", labels[v]);
        return;
    }

    printPath(parent, parent[v], labels);
    printf(" -> %c", labels[v]);
}

// Dijkstra's algorithm
Windsurf: Refactor | Explain | ×
void dijkstra(int graph[MAX][MAX], int n,
              int source, char labels[]) {
    int dist[MAX], visited[MAX], parent[MAX];

    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }

    dist[source] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1;
        int minDist = INF;

        // Find the unvisited vertex with minimum distance
        for (int i = 0; i < n; i++) {
            if (!visited[i] && dist[i] < minDist) {
                minDist = dist[i];
                u = i;
            }
        }

        if (u == -1)
            break;

        visited[u] = 1;

        // Relax all outgoing edges from u
        for (int v = 0; v < n; v++) {
            if (graph[u][v] > 0 && !visited[v] &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
```

Screenshot 1: Dijkstra’s path-printing function and main algorithm logic.

Screenshot 2

```
20                  int source, char labels[]) {
59      printf("\nSource Vertex: %c\n", labels[source]);
60      printf("\nVertex\tDistance\tShortest Path\n");
61
62      for (int i = 0; i < n; i++) {
63          printf("%c\t", labels[i]);
64
65          if (dist[i] == INF) {
66              printf("INF\t\tNo path\n");
67          } else {
68              printf("%d\t\t", dist[i]);
69              printPath(parent, i, labels);
70              printf("\n");
71          }
72      }
73  }
74
    Windsurf: Refactor | Explain | Generate Function Comment | ×
75  int main(void) {
76      int graph[MAX][MAX] = {0};
77      int choice;
78
79      printf("DIJKSTRA'S SHORTEST PATH ALGORITHM\n");
80      printf("1. Undirected Graph (Source: 0)\n");
81      printf("2. Directed Graph (Source: A)\n");
82      printf("Enter your choice: ");
83
84      if (scanf("%d", &choice) != 1 ||
85          (choice != 1 && choice != 2)) {
86          printf("Invalid choice!\n");
87          return 1;
88      }
89
90      if (choice == 1) {
91          // Undirected graph: vertices 0 to 8
92          char labels[MAX] = "012345678";
93          int n = 9;
94
95          // Each edge is added in both directions
96          int edges[][3] = {
97              {0, 1, 4},
98              {0, 4, 8},
99              {1, 2, 8},
100             {1, 4, 11},
101             {2, 3, 7},
102             {2, 8, 2},
103             {2, 6, 4},
104             {3, 6, 14},
105             {3, 7, 9},
106             {4, 8, 7},
107             {4, 5, 1},
108             {8, 5, 6},
109             {5, 6, 2},
110             {6, 7, 10}
111         };
112
```

Screenshot 2: Main function and definition of the undirected graph and its weighted edges.

Screenshot 3

```
112
113         int edgeCount = sizeof(edges) / sizeof(edges[0]);
114
115         for (int i = 0; i < edgeCount; i++) {
116             int u = edges[i][0];
117             int v = edges[i][1];
118             int w = edges[i][2];
119
120             graph[u][v] = w;
121             graph[v][u] = w;
122         }
123
124         printf("\nUNDIRECTED GRAPH\n");
125         dijkstra(graph, n, 0, labels);
126     } else {
127         // Directed graph: A=0, B=1, C=2, D=3, E=4
128         char labels[MAX] = "ABCDE";
129         int n = 5;
130
131         // Add only the directed edges shown in the diagram
132         graph[0][1] = 10;  // A -> B
133         graph[1][3] = 1;   // B -> D
134         graph[2][1] = 3;   // C -> B
135         graph[0][2] = 5;   // A -> C
136         graph[2][3] = 9;   // C -> D
137         graph[2][4] = 2;   // C -> E
138         graph[4][3] = 6;   // E -> D
139         graph[4][0] = 2;   // E -> A
140
141         printf("\nDIRECTED GRAPH\n");
142         dijkstra(graph, n, 0, labels);
143     }
144
145     return 0;
146 }
147
```

Screenshot 3: Completion of the graph setup for the undirected case and the directed graph edge definitions.

Screenshot 4

```
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa> gcc exp5.c -o exp5.exe
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa> .\exp5.exe
DIJKSTRA'S SHORTEST PATH ALGORITHM
1. Undirected Graph (Source: 0)
2. Directed Graph (Source: A)
Enter your choice: 1

UNDIRECTED GRAPH

Source Vertex: 0

Vertex  Distance        Shortest Path
0       0               0
1       4               0 -> 1
2       12              0 -> 1 -> 2
3       19              0 -> 1 -> 2 -> 3
4       8               0 -> 4
5       9               0 -> 4 -> 5
6       11              0 -> 4 -> 5 -> 6
7       21              0 -> 4 -> 5 -> 6 -> 7
8       14              0 -> 1 -> 2 -> 8
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa>  gcc exp5.c -o exp5.exe
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa> .\exp5.exe
DIJKSTRA'S SHORTEST PATH ALGORITHM
1. Undirected Graph (Source: 0)
2. Directed Graph (Source: A)
Enter your choice: 2

DIRECTED GRAPH

Source Vertex: A

Vertex  Distance        Shortest Path
A       0               A
B       8               A -> C -> B
C       5               A -> C
D       9               A -> C -> B -> D
E       7               A -> C -> E
PS C:\Users\Vaibhav Upadhyaya\OneDrive\Desktop\daa>
```

Screenshot 4: Successful terminal execution for both graphs, showing shortest distances and paths.